Capstone Project Management System: Design and Antigravity Build Pipeline

Oct 11, 2026 · @Dr. Susovan Mandal

The Departmental Project Committee will run one database-driven web app, hosted free on Vercel, where faculty pick or propose capstone topics, an AI tags each topic against WP, WK, EA, SDG and PO, supervisors enter marks, and the committee reads off which projects earn capstone status. This document is the full design and the prompt-by-prompt pipeline to build it in Google Antigravity.

1. Vision, roles and scope

The app removes the manual scorecard burden from faculty: they choose or write a topic and a short description, and the system proposes every mapping. The faculty member only confirms. The committee then sees attainment automatically from the marks.

Role	Who	Can do	Cannot do
Admin	Project Committee members	Import topics (CSV/Excel), configure AI providers and criteria, upload groups and student data, see all attainment, track missing marks, send reminders, export lists, declare capstones	Propose or submit a project topic
Faculty (supervisor)	Every teacher guiding projects	Search the topic bank, select or propose a topic, write or AI-generate the description, review and confirm mappings, see allotted groups, enter marks, see attainment of their own groups as pie charts, download rubric PDFs	See other supervisors' groups or any admin screen

Role	Who	Can do	Cannot do
Student	Not a login in version 1	Receives the exported topic list and allotment as PDF	(no account)

Rules taken from your description and fixed in the design:

- **Admin cannot submit.** A committee member who also supervises needs a second, separate faculty account. The database enforces one role per account, so the rule cannot be bypassed from the screen.
- **A topic goes live only after the faculty member confirms** the final description and mapping. AI output is always a suggestion.
- **Marks are entered by supervisors** for the 6th, 7th and 8th semesters. Rubrics B, D and F are scored by examiners, so the design lets the supervisor enter those scores from Forms 7-P and 8-E, and records who entered them. Open decision: do examiners need their own login later? (Section 10.)
- **Attainment is private by role.** A supervisor sees only their own groups. The admin sees every topic and student.
- **Student data is personal data.** Name, roll number, phone and email are stored under India's DPDP Act 2023 duties (Section 8).

2. Feature specification by role

Every feature below traces to a sentence in your brief. IDs (A1, F1, ...) are used later in the Antigravity prompts and the test plan.

Admin console

ID	Feature	Detail
A1	Topic import	Upload CSV or Excel; map columns to fields; preview, validate, de-duplicate; the two sheets of *SIH2026 and UG research SDG mapping v2* import as they are. UG rows have no Software/Hardware column, so the importer proposes one by keyword and the admin corrects it.

ID	Feature	Detail
A2	Criteria settings	Edit the qualification thresholds, Good-band cut-offs, attainment targets and the WK-to-PO map. Defaults are the department's values from the proposal.
A3	AI console	Add, test, rank and switch AI providers and keys (Section 6).
A4	Semester and batch setup	Academic year, semester, rubric set (A to F) and marking windows with deadlines.
A5	Faculty accounts	Invite supervisors by email, set role, deactivate.
A6	Allocation upload	Upload group number, student details (name, college roll, phone, email), topic code and supervisor; the app validates every link and rejects orphan rows.
A7	Marks tracker	Table of supervisors against semester, showing who has not uploaded; one click sends a reminder.
A8	Automatic reminders	Scheduled emails to supervisors with missing marks, with a daily or weekly cadence set by the admin (Section 8).
A9	Attainment views	By topic, group, student, supervisor, semester: POs, WPs, WKs, EAs, SDGs.
A10	Capstone list	Applies the department's capstone rule to scores and declarations, shows which rule step each project passes or fails, and lets the admin confirm the list.
A11	Exports	Approved topic list for students and the capstone list as Excel, CSV and PDF.
A12	Audit log	Who changed which mapping, mark or setting, and when.

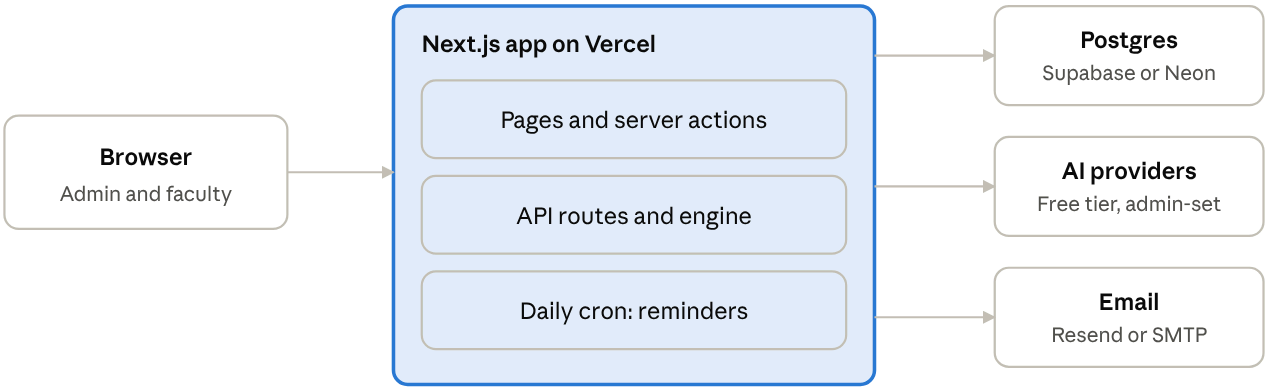
Faculty workspace

ID	Feature	Detail
F1	Topic search	Filter by SDG, category, Software or Hardware, domain, source (SIH or UG research); keyword search over title, description and tags; sort by relevance or SDG.
F2	Select or propose	Select a bank topic, or type your own title.
F3	Description	A box limited to 500 words with a live counter. Write it by hand, or enter a few keywords and press **Draft with AI**.
F4	Mapping panel	Shows WP1 to WP7, WK1 to WK9, EA1 to EA5, SDGs with targets, the POs reached, the qualification result and the likelihood of capstone status, each with a one-line reason.
F5	Edit the mapping	Switch any attribute on or off, or change its level; the AI then rewrites the description to match, and the faculty confirms before it replaces the text.
F6	Confirm	A final review screen; confirming publishes the topic with its code, description and SDG tag into the allotment list.
F7	My groups	After the admin uploads allocation, each supervisor sees their groups with full student details.
F8	Rubric PDFs	Download the rubric sheet for the examiner (A to F, per semester).
F9	Marks entry	Per group and per student, per criterion, with range checks against the rubric maximum, save as draft, then submit.
F10	My attainment	Pie charts of how much of each metric (PO, WP, WK, EA, SDG) the supervisor's own groups attained. Visible only to that supervisor.

3. Architecture and free-tier tech stack on Vercel

One Next.js application serves the screens and the API, talks to one Postgres database, and calls out to an AI provider and an email service. Nothing else needs a server, so it fits Vercel's free model. The stack follows what you already use in EduSphere (Next.js, Supabase, Prisma, Auth.js), so you can reuse that experience.

One Vercel app talks to the database, an AI provider and email



Student details go only to the database and to email. The AI receives topic text alone.

architecture · 1 app, 3 outside services

The browser reaches only the Next.js app. The app alone holds the database connection, the AI keys and the email credentials.

Layer	Choice	Why
App	Next.js 14+ (App Router), TypeScript	Pages and API routes in one project; deploys to Vercel with no configuration
UI	Tailwind CSS + shadcn/ui	Fast, accessible tables, dialogs and forms
Database	Supabase Postgres (or Neon) through Prisma	Relational data fits groups, students, marks and mappings; use the pooled connection string on Vercel
Auth	Auth.js with invited email and password; optional Google sign-in limited to the institute domain	Admin invites faculty; no public sign-up

Layer	Choice	Why
Import and export	Papa Parse (CSV), SheetJS or ExcelJS (Excel), @react-pdf/renderer (PDF)	Covers the CSV, Excel and PDF exports you asked for
Charts	Recharts	Pie charts for supervisors, bars and heat maps for the admin
AI	Provider-agnostic adapter over OpenAI-compatible endpoints (for example Google Gemini, Groq, OpenRouter)	Admin chooses and switches providers without code changes (Section 6)
Email	Resend free tier, or Gmail SMTP with an app password through Nodemailer	Reminder emails for missing marks
Scheduling	Vercel Cron calling a protected route	Daily reminder run
Validation and tests	Zod, Vitest, Playwright	Same schemas on the server and the form; end-to-end checks of each role

Points to settle before you deploy, because free-plan terms change and I could not verify today's limits:

- **Vercel Hobby cron** has been limited to one run per day, which is enough for reminders. Confirm the current limit.
- **Hobby plan terms** are for non-commercial personal use. A college department may need the institute's team account, or a Pro plan. Confirm with Vercel's terms.
- **Request time limits** on the free plan are short, so any slow job (large import, AI call) must be quick or split into chunks.
- **Free AI tiers** have rate limits and some may use prompts to improve their models. For that reason the app sends the AI only topic text, never student data.
- **Database free tiers** pause or cap storage; the volumes here (hundreds of topics, a few hundred students) are small, but check the current caps.

4. Database design

The schema keeps three things apart: the catalogue (POs, WK-PO map, WP/WK/EA wording, rubrics), the topics and their mappings, and the people and marks. The catalogue is data, not code, so the department can correct wording or thresholds from the admin console without a redeploy.

Group	Tables	Purpose
People	User, Student	Admin and faculty logins; student records (name, roll, phone, email)
Catalogue	ProgramOutcome (11), Attribute (WP1-7, WK1-9, EA1-5), WkPoMap, Sdg (17) and SdgTarget	The department's own PO and WK wording and mapping, seeded from your 2025-26 document
Rubrics	Rubric (A-F), Criterion, CriterionMap	Marks, maxima and the link from each criterion to WP, WK, EA, PO
Topics	Topic, TopicSdg, TopicAttribute, DescriptionVersion, ComplexityDeclaration	One row per topic; mappings carry level 0-3, a reason and an origin (rule, AI or faculty)
Allocation	Batch, Group, GroupMember	Group number, topic code, supervisor, members
Marks	MarkWindow, MarkEntry	Scores per criterion, per group or per student, draft or submitted
Results	AttainmentSnapshot, CapstoneDecision	Cached attainment and the capstone outcome with the rule steps passed
Operations	AiProvider, AiCallLog, ReminderLog, AuditLog, Setting	AI keys (encrypted), usage, reminders, change history

Core of the Prisma schema (trimmed to what drives the logic):

```
enum Role { ADMIN FACULTY }
enum TopicSource { SIH UG_RESEARCH FACULTY }
enum TopicStatus { DRAFT CONFIRMED ALLOTTED ARCHIVED }
enum Kind { WP WK EA }
enum Origin { RULE AI FACULTY }
```

```
model User {
  id String @id @default(cuid())
  email String @unique
  name String
  passwordHash String?
  role Role
  active Boolean @default(true)
  topics Topic[] @relation("proposer")
  groups Group[]
}

model Topic {
  id String @id @default(cuid())
  code String @unique            // SIH26001, UGR-017, FAC-2026-031
  title String
  source TopicSource
  category String                // Software | Hardware | Both
  domain String?
  organization String?
  description String?            // max 500 words, enforced in Zod
  tags String[]
  status TopicStatus @default(DRAFT)
  proposerId String?
  proposer User? @relation("proposer", fields: [proposerId], references:
[id])
  sdgs TopicSdg[]
  attributes TopicAttribute[]
  versions DescriptionVersion[]
  groups Group[]
}

model Attribute { id Int @id @default(autoincrement()) kind Kind code String
@unique title String text String }

model TopicAttribute {
  topicId String
  attributeId Int
  level Int                      // 0..3
  reason String
  origin Origin
  @@id([topicId, attributeId])
}
```

```
model WkPoMap { wk String po String @@id([wk, po]) }

model Criterion {
  id Int @id @default(autoincrement())
  rubric String                  // A..F
  semester Int                   // 6,7,8
  key String
  label String
  maxMarks Int
  scope String                   // GROUP | STUDENT
  maps CriterionMap[]
}
model CriterionMap { criterionId Int kind String code String
@@id([criterionId, kind, code]) } // kind: WP|WK|EA|PO

model Group {
  id String @id @default(cuid())
  number Int
  batch String
  topicId String
  supervisorId String
  members GroupMember[]
}
model Student { id String @id @default(cuid()) name String roll String
@unique phone String? email String? }
model GroupMember { groupId String studentId String @@id([groupId,
studentId]) }

model MarkEntry {
  id String @id @default(cuid())
  criterionId Int
  groupId String
  studentId String?              // null for group-level criteria
  marks Float
  status String                  // DRAFT | SUBMITTED
  enteredById String
  @@unique([criterionId, groupId, studentId])
}

model AiProvider {
  id String @id @default(cuid())
```

```
  name String baseUrl String model String
  encryptedKey String            // AES-256-GCM, master key in env
  priority Int enabled Boolean @default(true)
}
```

Indexes to add: Topic(source, category), TopicSdg(sdg), Group(supervisorId, batch), MarkEntry(groupId). A full-text index on Topic(title, description, tags) powers the keyword search in F1.

5. Mapping, attainment and capstone engine

This is the part that makes the app useful to NBA. It is written as pure functions with no database or AI calls inside, so the committee can read, test and defend every number. All thresholds below are defaults taken from the proposal and are editable in the admin console (A2).

5.1 At topic stage: the scorecard and the prediction

1. The system proposes a level from 0 to 3 for each of WP1 to WP7, WK1 to WK9 and EA1 to EA5, each with a one-line reason. Keyword rules give a baseline; the AI refines it (Section 6). The faculty member can change any level.
2. **Qualification rule:** WP1 at least 2; at least three of WP2 to WP7 at least 2; at least two of EA1 to EA5 at least 2; at least one SDG with a target named.
3. **PO potential:** each PO that has WK codes in the department's mapping gets the mean of its linked WK levels, shown as Low (under 1), Medium (1 to under 2) or High (2 or more). PO8, PO9 and PO10 have no WK link in the department document, so the app shows them as *assessed by rubric* and does not predict them.
4. **Capstone outlook:** *Not yet qualifying* if the rule fails; *Qualifies* if it passes; *Strong* if it passes with WP1 at 3, four or more of WP2 to WP7 at 2 or more, and three or more EA at 2 or more. The word *Strong* is a screening label, not a promise; the final decision uses marks (5.3).

5.2 Department WK-PO map seeded into the catalogue

PO	Linked WK
PO1 Engineering Knowledge	WK1, WK2, WK3, WK4
PO2 Problem Analysis	WK1, WK2, WK3, WK4
PO3 Design/Development of Solutions	WK5

PO	Linked WK
PO4 Conduct Investigations	WK8
PO5 Engineering Tool Usage	WK2, WK6
PO6 The Engineer and The World	WK1, WK5, WK7
PO7 Ethics	WK9
PO8 Teamwork, PO9 Communication, PO10 Project Management and Finance	none in the document
PO11 Life-Long Learning	WK8

5.3 After marks: attainment and the capstone decision

Each rubric criterion carries its own mapping to WP, WK, EA and PO (table CriterionMap), seeded from section 8.1 of the proposal. For any metric X:

```
Attainment(X) = \frac{\sum m_c}{\sum M_c} \quad \text{over criteria } c
\text{ mapped to } X
```

where m is the mark obtained and M the maximum. A metric counts as *attained* when the value reaches the target (default 60%, set by the department). The result is computed per student, per group, per supervisor, per topic, per semester and over the whole 6th to 8th sequence.

Two limits are shown on screen, not hidden:

- **WP5 and WP7 have no rubric criterion.** Their status comes from the complexity declaration and is labelled *declared*, never *attained*.
- **SDG attainment** uses the impact-on-society and realistic-problem criteria as evidence for the topic's SDGs, since marks are not given per SDG.

Capstone rule (proposal section 8.3), evaluated automatically and confirmed by the admin:

Step	Test	Default
1	Scorecard meets the qualification rule	Section 5.1
2	7th semester panel average, Rubric D	Novelty at least 5 of 7, realistic problem at least 5 of 7, impact at least 2 of 3

Step	Test	Default
3	8th semester external examiner, Rubric F	Novelty at least 6 of 8, realistic problem at least 6 of 8, impact at least 2 of 3
4	Complexity declaration confirms the attributes the rubrics do not assess	WP5, WP7 and, where claimed, WP2, WP6, EA1, EA2

The capstone screen shows each project's result on all four steps, so the committee sees why a project passed or missed. A project with missing marks shows *pending*, not *failed*.

Seed mapping of criteria (from proposal section 8.1; marks per the evaluation scheme PDF)

Criterion	Where	WP	WK	EA	PO
Literature survey	6th A, B	WP1, WP4	WK1, WK8	EA5	PO2, PO4, PO11
Knowledge of own work	6th B	WP1, WP3	WK3, WK4	none	PO1, PO2
Viva voce	6th B; 7th D; 8th F	WP1, WP3	WK3, WK4, WK6	none	PO1, PO9
Novelty / originality	7th D; 8th F	WP3, WP4	WK8	EA3, EA5	PO3, PO4
Realistic problem	7th D; 8th F	WP1, WP2	WK5, WK7	EA2	PO2, PO3, PO6
Impact on society	7th D; 8th F	WP6	WK7	EA4	PO6
Publication	7th D; 8th F	none	WK8	EA3	PO4, PO9, PO11
Presentation	6th B; 7th D; 8th F	none	none	none	PO9
Leadership and teamwork	7th C; 8th E	none	WK9	EA1	PO8, PO10
Behaviour and conduct	6th A; 7th C; 8th E	none	WK7, WK9	none	PO7, PO8
Regularity, task completion, involvement	6th A; 7th C; 8th E	none	none	none	PO10, PO11

The engine core, as the Antigravity agent should implement it in `lib/engine.ts`:

```
export type Levels = Record<string, number>; // 'WP1'..'WP7','EA1'..'EA5' ->
0..3

export function qualifies(l: Levels, hasSdgTarget: boolean, cfg =
DEFAULT_CFG) {
  const wp = ['WP2','WP3','WP4','WP5','WP6','WP7'].filter(k => (l[k] ?? 0) >=
cfg.minLevel).length;
  const ea = ['EA1','EA2','EA3','EA4','EA5'].filter(k => (l[k] ?? 0) >=
cfg.minLevel).length;
  return (l.WP1 ?? 0) >= cfg.minLevel && wp >= cfg.minWpOfSix && ea >=
cfg.minEa && hasSdgTarget;
}

export function poPotential(wkLevels: Levels, wkPo: Record<string, string[]>)
{
  // mean of linked WK levels per PO; POs with no WK link return null
(assessed by rubric)
  return Object.fromEntries(Object.entries(wkPo).map(([po, wks]) =>
    [po, wks.length ? wks.reduce((s, k) => s + (wkLevels[k] ?? 0), 0) /
wks.length : null]));
}

export function attainment(entries: {marks:number; max:number; maps:string[]}
[], metric: string) {
  const e = entries.filter(x => x.maps.includes(metric));
  const M = e.reduce((s, x) => s + x.max, 0);
  return M === 0 ? null : e.reduce((s, x) => s + x.marks, 0) / M;
}
```

One schema detail the code must respect: Rubric D is scored by a panel, so MarkEntry needs an `examinerSlot` field in its unique key, and step 2 averages the slots.

6. AI layer

The AI does two jobs only: write a description from keywords, and propose a mapping with reasons. It never confirms anything. The admin decides which free AI services the app may use.

Provider adapter. Most free AI services (Google Gemini, Groq, OpenRouter) accept the OpenAI-style `/chat/completions` request, so one adapter covers them. Each provider is a row in AiProvider: name, base URL, model, key, priority, on/off. The admin console (A3)

has these actions:

- Add a provider and paste its key; the key is encrypted with AES-256-GCM before it is stored, and the master key sits in a Vercel environment variable.
- **Test** button: sends a tiny prompt and shows success, delay and error text.
- Drag to set priority. If the first provider fails or hits its rate limit, the app tries the next, then falls back to rule-based tagging so the faculty member is never blocked.
- A daily call limit per provider and per faculty member, with usage shown from AiCallLog.

Three prompts; the mapping prompt returns JSON that Zod validates.

Prompt 1, Draft description. Input: title, category, SDG hint, up to ten keywords. Output: a description of at most 500 words covering problem, approach, expected outcome and the SDG link. The server cuts anything beyond 500 words and shows the counter.

Prompt 2, Map the topic. Input: title and description, plus the catalogue wording of WP1-7, WK1-9, EA1-5 and the 17 SDGs. Output:

```
{
  "attributes": [{"code": "WP1", "level": 3, "reason": "Requires full-wave EM
analysis of ..."}],
  "sdgs": [{"sdg": 7, "target": "7.2", "reason": "..."}],
  "missing": ["WP5", "WP7"],
  "suggestions": ["Add a trade-off between cost and efficiency to raise
WP2."]
}
```

Prompt 3, Rewrite to match. When the faculty member changes the mapping, the AI gets the current description and the new target levels and returns a revised description. The screen shows old and new side by side and nothing is replaced until the faculty member presses **Accept**.

Guardrails

- The prompt carries only topic text. No student names, rolls, phones or emails ever reach an AI provider.
- Output is validated against the schema; an invalid reply is retried once, then discarded.
- Each mapping line shows its origin: *rule*, *AI* or *faculty*, and any change by the faculty member is logged.
- Where the AI and the keyword rules disagree by two or more levels, the line is flagged for the faculty member's attention.

- The system prompt tells the model to use only the attribute wording supplied and to say "insufficient information" instead of guessing.
- Free tiers can change or vanish, which is why the fallback and the multi-provider list matter.

7. Screens and user flows

Faculty flow, from search to confirmed topic

1. Sign in and open **Find a topic**. Filter by SDG, category, Software or Hardware, domain or keyword.
2. Select a topic, or press **Propose my own** and type a title.
3. On **Describe**, write up to 500 words, or enter keywords and press **Draft with AI**. Edit freely.
4. Press **Analyse**. The **Mapping** screen shows WP, WK, EA and SDG with levels and reasons, the PO potential, the qualification result and the capstone outlook.
5. To change a mapping, move a level. A **Rewrite description** button asks the AI for new wording; the faculty member compares and accepts or rejects it.
6. On **Review and confirm**, the topic, description, SDG tag and mapping appear on one page. Confirming publishes it with a topic code.
7. After allotment, **My groups** lists the groups; **Marks** opens the entry grid; **My attainment** shows the pie charts.

Screen map

Area	Screen	Main content
Public	Sign in, Accept invite, Reset password	Email and password; no self-registration
Faculty	Find a topic	Filter bar, result cards with SDG chips, Software/Hardware badge, source badge
Faculty	Describe	Rich text box, live word counter (max 500), AI draft panel
Faculty	Mapping	Four tabs or cards: WP, WK, EA, SDG; PO potential bars; qualification checklist with ticks; capstone outlook

Area	Screen	Main content
Faculty	Review and confirm	Read-only summary and a **Confirm** button
Faculty	My topics	Status of each topic: draft, confirmed, allotted
Faculty	My groups	Group cards with member table (name, roll, phone, email)
Faculty	Marks	Grid per semester and rubric; validates each cell against its maximum; draft and submit
Faculty	Rubric PDFs	Download A to F as examiner sheets
Faculty	My attainment	Pie charts per PO, WP, WK, EA, SDG for own groups; filter by semester
Admin	Dashboard	Counts of topics, allotments, marks received, capstone candidates
Admin	Topic bank	Import wizard, table with filters, edit, archive
Admin	Confirmed topics	Review, edit, export for students (Excel, CSV, PDF)
Admin	Allocation	Upload groups and students; validation report; link groups to topic code and supervisor
Admin	Marks tracker	Supervisor by semester matrix, status chips, **Remind** button, reminder history
Admin	Attainment	Pick topic, group, student or supervisor; heat map of POs and WP/WK/EA against groups
Admin	Capstone	Table with the four rule steps per project, filters, confirm and export
Admin	Statistics	Topic-wise and student-wise tables and charts, SDG spread, Software against Hardware, semester trends
Admin	Settings	Criteria, WK-PO map, rubrics, AI providers, email, reminder cadence
Admin	Audit log	Searchable change history

Charts, by who sees them

- *Supervisor:* pie charts of attained against not-attained share for each metric, for own groups only.
- *Admin:* the same plus a group-by-metric heat map, a ranked bar chart of capstone candidates and an SDG coverage chart.
- Pie charts suit the supervisor's view because each metric is a share of one whole. For the admin's comparisons across many groups, bars and heat maps read better than many pies.

8. Reminders, exports, security and privacy

Automatic reminders (A7, A8)

- Each marking window (semester, rubric, deadline) has a reminder policy set by the admin: first reminder some days before the deadline, then every few days, then daily once overdue, with a maximum count.
- A Vercel Cron job calls a protected route once a day (set to the morning in IST). The route compares each supervisor's groups with submitted marks and emails only those with missing entries.
- The email names the semester and the groups pending, and links to the Marks screen. The admin can also press **Remind** on a row at any time.
- Every send is written to ReminderLog, so the tracker shows the last reminder and the count, and a failed send is visible.
- The cron route accepts only requests carrying the secret that Vercel supplies in the environment variable `CRON_SECRET`.

Exports (A11, F8)

Export	Formats	Contents
Approved topic list for students	Excel, CSV, PDF	Topic code, title, category, SDG tags, description, supervisor name; no student data
Allocation sheet	Excel, PDF	Group, topic code, supervisor and members; for the department file

Export	Formats	Contents
Capstone list	Excel, CSV, PDF	Project, group, supervisor, the four rule steps, final status
Attainment report	Excel, PDF	PO, WP, WK, EA, SDG attainment by topic, group, student
Rubric sheets	PDF	Rubrics A to F as printable examiner forms

Cells that begin with `=`, `+`, `-` or `@` are prefixed with an apostrophe in CSV and Excel exports, to stop spreadsheet formula injection.

Security

- **Roles are enforced on the server.** Every API route and server action checks the role, and every supervisor query is filtered by the signed-in user's id, so a faculty member cannot reach another supervisor's groups by changing a URL.
- Passwords hashed with Argon2 or bcrypt; invites expire; sign-in attempts are rate limited; cookies are secure and HTTP-only.
- AI keys encrypted at rest and never sent to the browser. Secrets live only in Vercel environment variables.
- Marks lock when a window closes; only an admin can reopen one, and the audit log records it.
- The Supabase service key is used only on the server. Browsers never talk to the database directly.
- A weekly database export is kept by the admin as a backup.

Privacy of student data

The app holds names, roll numbers, phones and emails, which is personal data under India's Digital Personal Data Protection Act, 2023. Practical steps: tell students what is collected and why, use it only for project allocation and evaluation, show contact details only to the group's supervisor and the admin, never send it to an AI provider, and delete or anonymise contact fields after the retention period the institute sets. Check the institute's data policy before going live.

9. Antigravity build pipeline

The build runs as 13 small phases. Each phase has one prompt to paste into an Antigravity agent, a list of checks you run before moving on, and a git commit. Small phases keep the agent's plan reviewable, and a mistake costs one phase, not the whole app. Antigravity's menu names and its rules and workflows folders can differ between versions, so check its Customizations panel for where project rules live.

Phase 0. Preparation (you, about an hour)

1. Create an empty Git repository on GitHub and open it as a workspace in Antigravity.
2. Create a free Supabase (or Neon) project and copy its pooled and direct connection strings.
3. Create a Vercel project linked to the repository. Add a Resend account, or a Gmail app password, for email.
4. Get one free AI key (for example from Google AI Studio) for testing. More can be added later in the admin console.
5. Make a `/seed` folder with: the topic workbook *SIH2026_and_UG_Research_SDG_Mapping_v2.xlsx*, the *Project Evaluation Scheme* PDF, the *PO_PSO_etc_B.Tech_2025-26* document and the proposal .docx. Text copies of the PDF and the PO document help the agent read them without parsing errors.
6. Generate secrets: `AUTH_SECRET`, `APP_ENCRYPTION_KEY` (32 random bytes, base64) and `CRON_SECRET`.

Project rules (paste into the workspace rules file, for example `.agent/rules/project.md`, and into `AGENTS.md`)

```
Project: ECE Capstone Project Management System for HITK (B.Tech ECE).
Stack: Next.js 14+ App Router, TypeScript strict, Tailwind, shadcn/ui, Prisma
+ Postgres (Supabase), Auth.js (credentials, invite-only), Zod, Recharts,
@react-pdf/renderer, ExcelJS, Papa Parse, Vitest, Playwright. Deploy target:
Vercel.
Roles: ADMIN (project committee) and FACULTY. Admin can never create or edit
a topic as proposer. One role per account.
Rules:
1. Every route, server action and API handler checks role on the server.
Faculty queries are always filtered by the signed-in user's id.
2. Never send student names, roll numbers, phones or emails to any AI
provider. AI input is topic text only.
3. All thresholds, the WK-PO map, rubrics and criterion maps live in the
database (seeded), not hard-coded. Defaults come from /seed documents.
4. Engine logic (qualification, PO potential, attainment, capstone rule) is
in lib/engine as pure functions with unit tests.
5. AI output is always validated with Zod and is only a suggestion; the
faculty member confirms.
6. API keys are stored encrypted (AES-256-GCM). Never log secrets or return
them to the browser.
7. Use environment variables: DATABASE_URL, DIRECT_URL, AUTH_SECRET,
APP_ENCRYPTION_KEY, RESEND_API_KEY, MAIL_FROM, CRON_SECRET, APP_BASE_URL.
8. Keep requests short for Vercel's free plan: chunk imports, paginate lists.
9. Write a plan artifact first; make small commits; run lint, typecheck and
tests before declaring a phase done; do not change the schema without saying
so in the plan.
10. Do not invent department data. If a value is missing from /seed, create a
clearly marked TODO and ask.
```

Phase 1. Scaffold, auth and roles

```
Create the Next.js app with the stack in the project rules. Implement Auth.js
credentials sign-in, invite-only sign-up (admin creates an invite link with
an expiring token; the invitee sets a password), password reset by email,
role-based middleware, and a shell layout with a sidebar that differs for
ADMIN and FACULTY. Seed one admin from environment variables. Add a /healthz
route. Write Playwright tests: faculty cannot open /admin/*, admin cannot
open the Propose screens, unauthenticated users go to sign-in.
```

Checks: sign in as admin and as a test faculty; hitting an admin URL as faculty returns 403; the tests pass.

Phase 2. Schema and catalogue seed

```
Implement the Prisma schema from section 4 of the design document, plus
examinerSlot on MarkEntry. Write prisma/seed.ts that loads from /seed: (a)
the 11 POs and the WK1-WK9 wording exactly as in the PO document, (b) the WK-
PO map from that document, (c) WP1-WP7 and EA1-EA5 from the proposal, (d) the
17 SDGs with targets, (e) Rubrics A-F with every criterion, maximum marks and
scope (GROUP or STUDENT) read from the Project Evaluation Scheme, (f)
CriterionMap rows from section 5.3 of the design document. Print a
verification report listing counts and any criterion whose marks could not be
read. Do not guess marks.
```

Checks: 11 POs, 9 WK, 7 WP, 5 EA, 17 SDGs; the rubric maxima add up to 60, 40, 50, 50, 40, 60 for A to F; you spot-check five criteria against the PDF.

Phase 3. Admin: topic import wizard

```
Build /admin/topics/import. Accept CSV or XLSX. Step 1 upload, step 2 map
columns to Topic fields (auto-detect from headers), step 3 preview with
validation errors and duplicates (same title or code), step 4 commit in
chunks of 100. Support both sheets of
SIH2026_and_UG_Research_SDG_Mapping_v2.xlsx: for SIH rows use PS Number as
code, Category as category, Theme as domain, Mapped SDG(s) to TopicSdg; for
UG research rows generate codes UGR-001.., set category by keyword (hardware
words: fabrication, antenna, sensor, FPGA, PCB; otherwise software) and let
the admin edit it in the preview. Store keywords in tags. Add the topic bank
table with filters and inline edit.
```

Checks: 233 SIH and 65 UG topics import; re-import creates no duplicates; the UG category guesses are editable before commit.

Phase 4. Admin: AI console and provider adapter

```
Build lib/ai with a provider adapter over OpenAI-compatible
/chat/completions, with timeout, one retry, priority fallback across enabled
providers, per-provider daily limit and an AiCallLog. Build
/admin/settings/ai: add provider (name, base URL, model, key), encrypt the
key with AES-256-GCM, Test button, enable toggle, priority ordering, usage
table. Provide presets for Google Gemini (OpenAI-compatible endpoint), Groq
and OpenRouter that the admin can edit. Add unit tests with a mocked fetch
for fallback, timeout and invalid JSON.
```

Checks: add a real key, press Test, see success; disable it and see the fallback message; the key never appears in any network response.

Phase 5. Faculty: topic search

```
Build /faculty/topics: filter by SDG (multi-select), category (Software,
Hardware, Both), domain, source (SIH, UG research), keyword search using
Postgres full-text across title, description, tags; sort by relevance;
pagination; result cards with SDG chips and badges; a detail drawer; buttons
Select and Propose my own. Create the Topic row as DRAFT linked to the
proposer when selected or proposed. Seed 20 test topics if the database is
empty in development.
```

Checks: each filter narrows results correctly; a keyword like "antenna" returns UG antenna topics; searching stays fast with 300 topics.

Phase 6. Faculty: description, AI draft, mapping and confirm

```
Build the Describe, Mapping and Review screens. Describe: textarea with a
live word counter, hard limit 500 words, Draft with AI from up to ten
keywords (Prompt 1). Mapping: first run keyword rules in lib/rules to give
baseline levels for WP, WK, EA and SDG, then call the AI (Prompt 2) and
merge, flagging lines where rules and AI differ by 2 or more. Show every
attribute with level 0-3, reason and origin (rule, AI, faculty). Use
lib/engine to show the qualification checklist, PO potential (PO8-PO10 shown
as assessed by rubric), and the capstone outlook labels. When the faculty
member edits a level, enable Rewrite description (Prompt 3) with a side-by-
side diff and Accept or Reject. Review shows everything read-only; Confirm
sets status CONFIRMED, creates a DescriptionVersion, writes AuditLog, and
assigns a topic code for faculty-proposed topics (FAC-<year>-<seq>). If all
AI providers fail, fall back to rules only and say so on screen.
```

Checks: the 500-word limit cannot be bypassed; changing WP2 from 1 to 3 flips the qualification checklist; the AI-off path still works; nothing confirmed without the button.

Phase 7. Admin: confirmed topics and exports

```
Build /admin/confirmed: table of confirmed topics with filters, edit,
archive, and the ability to see mapping and description history. Add Export
for students: Excel, CSV and PDF with topic code, title, category, SDG tags,
description and supervisor name, no student data. Sanitise CSV and Excel
cells that start with = + - or @. PDF uses @react-pdf/renderer with the
department header, page numbers and an SDG legend.
```

Checks: the three exports open correctly; a title starting with `=` is neutralised; the PDF paginates 100 topics cleanly.

Phase 8. Admin: allocation upload and Faculty: My groups

```
Build /admin/allocation: upload CSV or XLSX with columns Group No, Student Name, College Roll, Phone, Email, Topic Code, Supervisor Email. Validate: topic code exists and is CONFIRMED, supervisor exists, a roll number belongs to one group per batch, group size within settings. Show a validation report and commit only valid rows, with a downloadable error file. On commit, set the topic status ALLOTTED. Build /faculty/groups showing only the signed-in supervisor's groups with member name, roll, phone, email. Add Playwright tests proving supervisor A cannot see supervisor B's groups by URL or API.
```

Checks: a bad topic code is rejected with a clear row number; the isolation test passes.

Phase 9. Rubric PDFs and marks entry

```
Build /faculty/rubrics to download Rubrics A-F as printable examiner sheets (criteria, maximum, space for marks) using the seeded rubric data. Build /faculty/marks: choose semester and rubric, then a grid per group (GROUP-scope criteria) and per student (STUDENT-scope criteria). Validate 0 <= marks <= maximum, support Save draft and Submit, lock after the window deadline, and let the admin reopen a window with an audit entry. Record enteredById and examinerSlot (for panel marks in Rubric D, allow up to N slots set in settings). Show a completeness bar per group.
```

Checks: totals of each rubric equal its stated maximum; a mark above the maximum cannot be saved; a locked window refuses edits.

Phase 10. Attainment and dashboards

```
Implement lib/engine attainment functions per section 5.3 for PO, WP, WK, EA and SDG, aggregated per student, group, supervisor, topic, semester and the whole sequence, with the target from settings. Cache results in AttainmentSnapshot, recomputed when marks are submitted. Build /faculty/attainment with pie charts (attained vs not attained share) per metric for the supervisor's own groups only. Build /admin/attainment with a group-by-metric heat map, ranked bars and filters, and /admin/statistics with topic-wise and student-wise tables, SDG coverage, Software vs Hardware, and semester trends. Label WP5 and WP7 as declared, not attained. Write unit tests with hand-computed examples.
```

Checks: a hand-worked example of one group matches the screen to two decimals; a supervisor cannot fetch another's attainment through the API.

Phase 11. Capstone identification

```
Implement the capstone rule from section 5.3 as lib/engine/capstone.ts with
the four steps and thresholds from settings. Add the ComplexityDeclaration
form for supervisors (attributes the rubrics do not assess, with a one-line
justification each) and the CapstoneDecision table. Build /admin/capstone:
per project show step 1-4 results (pass, fail, pending), explain failures,
allow the admin to confirm or hold, and export the list in Excel, CSV and
PDF. Unit-test every boundary (for example Rubric F novelty 5.9 vs 6 of 8).
```

Checks: boundary tests pass; a project with missing marks shows pending, not fail.

Phase 12. Marks tracker, reminders and email

```
Build /admin/marks-tracker: matrix of supervisors by semester with counts of
groups complete, partial and missing, and a Remind button. Implement the
reminder policy (first reminder N days before deadline, then every K days,
daily when overdue, maximum M) in settings. Add /api/cron/reminders protected
by Authorization: Bearer CRON_SECRET, scheduled once a day in vercel.json.
Send mail through Resend or SMTP with a plain, clear template listing pending
groups and a link. Write ReminderLog entries and show history. Add a dry-run
mode that lists who would be emailed without sending.
```

Checks: dry run lists the right supervisors; one real test email arrives; a call to the cron route without the secret returns 401.

Phase 13. Hardening, tests and release

```
Run a security review: role checks on every route, input validation, rate
limiting on sign-in and AI endpoints, no secrets in client bundles, CSV
injection tests, security headers. Add an audit log viewer. Add database
backup notes. Add seed data and a demo mode with fake students only. Run the
full Playwright suite for both roles, fix failures, then produce a
walkthrough artifact with screenshots of every screen. Prepare vercel.json,
environment variable documentation and a README for the committee.
```

Checks: the full test suite is green; you read the walkthrough and approve each screen; a fresh deployment works from the README alone.

How to work each phase in Antigravity

1. Open the Agent Manager, start a new task, and paste the phase prompt. Use Planning mode so the agent writes a plan first.
2. Read the plan and task list before the agent writes code. Comment on anything that invents department data or changes the schema.

3. Let it run, then read the walkthrough and screenshots it produces. Run the checks yourself in the browser.
4. If something is wrong, reply in the same task with the exact failing check; do not start a new task for a small fix.
5. Commit and push; Vercel builds a preview. Test the preview before starting the next phase.
6. Keep Phase 2's seed verification report: it is your evidence that the app uses the department's own PO, WK and rubric data.

10. Deployment, testing, risks and open decisions

Deploy on Vercel

1. Push the repository to GitHub and import it in Vercel; the framework is detected as Next.js.
2. Add the environment variables below for Production and Preview.
3. Run `prisma migrate deploy` against the production database once, then `prisma db seed`. Add `prisma generate` to the build command.
4. Put `vercel.json` with one daily cron entry pointing to `/api/cron/reminders`.
5. Create the admin account from the seed variables, sign in, change the password, add AI providers, import topics, invite faculty.

Variable	Purpose
DATABASE_URL, DIRECT_URL	Pooled connection for the app; direct connection for migrations
AUTH_SECRET	Signs sessions
APP_ENCRYPTION_KEY	Encrypts AI provider keys
RESEND_API_KEY, MAIL_FROM	Email sending
CRON_SECRET	Authorises the reminder job
APP_BASE_URL	Links inside emails

Test plan

Level	What	Tool
Unit	Qualification rule, PO potential, attainment, capstone boundaries, word limit, encryption round trip	Vitest
Integration	Import, allocation validation, marks locking, AI fallback with mocked providers	Vitest + test database
End to end	Each feature A1-A12 and F1-F10 as one scenario per role, plus isolation between supervisors	Playwright
Acceptance	One full cycle with five real topics, three supervisors and fake students: propose, confirm, allot, mark, attain, capstone	You and the committee

Risks and how the design handles them

Risk	Handling
Free AI tier is limited, changes terms or goes away	Several providers with fallback; rules-only mode; faculty can always type the mapping
AI over-claims complexity	Reasons shown per line, disagreement flags, faculty confirmation, and capstone status decided by marks, not by AI
Mapping and thresholds are the committee's own choices, not NBA rules	All values editable and recorded in the audit log; state this in the NBA file
Marks entered for examiners by supervisors	Entry recorded with who entered it; examiner logins can be added later
Free-plan limits on database, email or cron	Small data volumes; one daily cron; check limits before each academic year
Student data leak	Server-side isolation, no AI exposure, retention policy, audit log
Antigravity agent invents department data	Rule 10, the seed verification report, and your spot checks in Phase 2

Decisions I need from you or the committee

- Who enters Rubric B, D and F marks: supervisors on the examiners′ behalf (as designed), or examiners with their own logins?
- Attainment target: the default is 60 percent of the maximum; the department may use a different target level.
- Capstone outlook labels (*Qualifies*, *Strong*): keep my definition or change the thresholds?
- Should the admin-confirmed topic list show supervisor names to students, or only topic codes?
- Is the 500-word limit on the description fixed, and should the AI-written text be labelled as AI-assisted in the exported list?
- Whether the app uses the institute′s email domain for sign-in, or invited personal emails.
- How long student contact details are kept after the batch graduates.

What I could not verify. Free-tier limits, pricing and the Hobby plan terms of Vercel, Supabase, Resend and the AI services, and Antigravity′s current menu and rules-folder names. Check each before you rely on it. The PO, WK and rubric data come from your documents; the WP and EA wording follows the Washington Accord criteria as I understand them and should be checked against your NBA SAR.